

miniFlink ex07

Сервис локации и газовых алертов для подземной шахты:
пошаговый разбор архитектуры

Abstract

ex07 — самый большой пример в репозитории miniFlink. Он строит сервис для подземной угольной шахты, который из двух независимых потоков (пакеты от фонарей и пакеты от газовых датчиков) выдаёт три типа выходов: алерты состояния шахтёра, локацию по маякам связи и обогащённые координатами газовые алерты. Под капотом используется почти весь арсенал библиотеки: скользящие окна с `allowed_lateness`, `group_by`, FSM через `process_keyed`, объединение потоков с выравниванием `watermark`, `retract`-семантика для обновлений.

Эта статья — подробный разбор того, *зачем* каждое решение принято таким, как оно принято: какие альтернативы рассматривались, почему отброшены, что бы случилось при наивной реализации. Цель — дать инженеру, выбирающему stream processing, конкретный материал для оценки, а контрибьютору — понимание design-rationale.

Contents

1	Задача	1
1.1	Что должен делать сервис	2
1.2	Что в этой задаче сложного	2
1.3	Почему через библиотеку, а не «просто кодом»	3
1.4	Что есть готового в miniFlink	4
1.5	Что увидим дальше	5
2	Входы	5
3	Три пайплайна	5
4	Retract как сквозной механизм	5
5	Производительность	5
6	Выводы	5

1 Задача

Представьте себе подземную угольную шахту: километры тоннелей на нескольких горизонтах, в среднем по сорок-сто шахтёров на смену, у каждого фонарь, который раз в пятнадцать секунд шлёт радиопакет с показаниями нескольких ближайших маяков связи, а раз в десять секунд — отдельный пакет с показаниями газовых сенсоров. Эти пакеты летят в диспетчерскую через смесь UDP-радио и LTE-шлюзов, доходят с задержками, иногда дублируются, иногда теряются.

В диспетчерской на стене висит экран с картой шахты и состоянием смены: где находится каждый шахтёр, у кого тревога по газу, кто прижал кнопку SOS, у кого вот-вот сядет батарейка фонаря. Этот экран обслуживает наш сервис.

1.1 Что должен делать сервис

Если разложить требования *диспетчера* на конкретные пайплайны, получается три независимых подсистемы:

Контроль состояния шахтёра. Из RSSI-пакета извлекаем не показания маяков (это работа другой подсистемы), а *косвенные факты*: пакет вообще пришёл, в нём были показания, фонарь двигался, напряжение батареи в норме. По каждому из этих фактов держим независимый таймер и эмитим соответствующий алерт когда таймер перевалил порог: “не было пакетов больше двух минут”, “пакеты идут, но маяков не слышит больше пяти”, “не двигался дольше порога”, “батарея ниже 3,5 В с дребезгом по гистерезису”. Плюс импульс SOS (нажатая кнопка) — мгновенный без таймера.

Локация по маякам. Для каждого шахтёра в скользящем 60-секундном окне с шагом 5 секунд берём топ-2 маяков с самой сильной медианной RSSI и из них приближённо вычисляем координаты шахтёра через интерполяцию между двумя маяками пропорционально расстоянию. Окно скользит каждые 5 секунд, поэтому диспетчерский экран обновляется не реже чем раз в 5 с; при опаздавшем пакете окно *переоткрывается* и эмитит **Retract** старого значения с последующим обновлённым **Data** — так картинка остаётся консистентной.

Газовые алерты с обогащением координатами. Газовый пакет приходит *отдельно* и сам по себе не несёт ID маяков или позиции. Когда газ превышает порог — алерт идёт немедленно (безопасность важнее красоты), но без координат, если их ещё не было. Как только локация для этого шахтёра становится известна (хотя бы по одному маяку), алерт автоматически *обновляется*: старая версия без координат retract’ится, новая с координатами эмитится. То же самое при смене уровня опасности (Warning → Critical) или существенном изменении ppm. Когда газ возвращается в норму — алерт снимается через **Gas_resolved**.

Эти три подсистемы независимы по бизнес-логике, но обмениваются данными: газовый пайплайн *потребляет* результат пайплайна локации, чтобы обогатить свои алерты координатами.

1.2 Что в этой задаче сложного

Если бы это была учебная задача из курса по базам данных, её можно было бы решить за полчаса: батчевый ETL раз в минуту, JOIN по шахтёру, отрисовка в дашборд. Но в реальной шахте такое решение *бесполезно*, и причин этому несколько.

Время событий не равно времени поступления. Пакет от шахтёра M1 с временной меткой **t=88s** может прийти в диспетчерскую в момент **t=210s**: UDP в шахте теряет пакеты, LTE-шлюз делает повторные попытки, на агрегаторе очереди. Если мы будем считать алерты по wall-clock времени поступления, то получим “M1 не двигался последние 2 минуты” в момент когда M1 уже давно дома — его пакет о движении просто опаздывает по дороге. Сервис должен работать по *event-time* (времени самого события), а не по wall-clock.

Идемпотентность. Тот же UDP-механизм даёт дубликаты: пакет может прийти дважды, потому что шлюз ретранслировал его при сомнениях. Если мы будем считать тревогу по второму дублию “независимо”, получим *два* алерта SOS на одно нажатие кнопки. Диспетчер не должен видеть фантомных тревог.

Опоздавшие пакеты ломают окна. Окно [60s, 120s) закрылось в 120s, и пайплайн эмитнул финальное top-2 маяков. Но в 145s пришёл опоздавший пакет с `ts=90s` — он *принадлежит* уже закрытому окну. Нужно либо его выбросить (теряем данные), либо переоткрыть окно, пересчитать агрегат и эмитнуть новое значение, помечая старое как недействительное. Второе правильно для диспетчера; первое было бы дешевле, но даёт непредсказуемый дашборд (на одной и той же временной метке могут быть разные значения в разные моменты wall-clock).

Газовый алерт без координат лучше чем нет алерта. Газ начинает выделяться *раньше* чем шахтёр успевает прийти под маяк — особенно в забое, где маяки ставят редко. Если ждать координат прежде чем эмитить тревогу о 120 ppm CO, можно потерять минуты на ожидании. Алерт идёт сразу. Но как только координаты появляются — они должны *подтянуться* к уже существующей тревоге, не порождая дубль и не давая диспетчеру повод закрыть старый алерт и открыть новый вручную.

Объём. Шахта на сорок шахтёров — мелочь. Реальные большие шахты в Кузбассе или Караганде имеют до четырёх тысяч работников в смену. RSSI-пакетов от такой шахты получается порядка 270 ev/s, газовых — ещё 400 ev/s. Это немного по меркам стримов, но *пайплайн должен держать нагрузку с запасом*: пиковые моменты (несколько горящих участков, грозовая помеха в радио, перезапуск шлюза) дают взрывы в десятки тысяч событий в секунду на короткий период. Сервис не должен в эти моменты терять алерты.

1.3 Почему через библиотеку, а не «просто кодом»

Каждый описанный выше пункт сложности *можно* решить вручную: структуры данных в OCamI есть, потоки тоже, таймеры в Unix доступны, голый код пишется. Но есть несколько мест, где «можно вручную» означает *можно сделать так, что в проде это работать не будет*. miniFlink в каждом из них берёт на себя самую тонкую часть.

Event-time и watermark. Без явной модели событийного времени любая обработка с окнами и таймерами скатывается к *wall-clock*: «считаем средний RSSI за последнюю минуту по часам в нашем сервере». В шахте это даёт явные ошибки: пакет с `ts=88s` физически пришёл к сервису в 210s (UDP + LTE-retry + очередь), и попадает в окно для текущей минуты, искажая её. miniFlink требует чтобы в потоке шли явные **Watermark**-события и каждый оператор окон работал по ним. Голый аналог пришлось бы писать самому, причём правильно: проверять ordering, перепроверять для каждого нового пайплайна, тестировать для каждого варианта late-семантики.

Retract как первоклассное событие. `allowed_lateness` в `window_agg_keyed` даёт нам *автоматический* retract: опоздавший пакет, который попадает в уже закрытое окно (в пределах `allowed_lateness`), вызывает **пересчёт** окна и эмиссию **Retract** старого значения плюс новый **Data**. Голый код имеет два пути: либо просто дропать опоздавшие пакеты (теряем данные ради простоты), либо пересчитывать без retract'ов (диспетчер видит мигание значений на дашборде, потому что окна *переоткрываются* молча). Третий правильный путь — сделать retract как явное событие — это и есть то, что библиотека реализует один раз и для всех пайплайнов.

Дедупликация по бизнес-правилу. UDP-шлюз в шахте ретранслирует пакеты при сомнениях — получаем дубли. Наивное «HashSet по сериализованным байтам» сломается мгновенно: шлюз обогащает пакет полем `received_at`, и идентичные по смыслу пакеты

различаются на байтовом уровне. `Pipe.dedup` берёт правило вида `p.lamp ^ string_of_int p.ts` — ключевое поле бизнеса, не байты транспортного протокола.

State per-key + automatic cleanup. `Pipe.process_keyed` держит state per-key (по шахтёру), вызывает наш обработчик, поддерживает self-timer’ы для timeout- семантики (alert после двух минут тишины). На четырёх тысячах шахтёров голый код потребует свой `Hashtbl<lamp, state>`, ручную регистрацию таймеров, ручную очистку state после конца смены. Каждое из этого — источник багов; например, мы могли бы накопить state шахтёров прошлой смены и тек уть память. Библиотека предоставляет TTL-state из коробки.

Channel с backpressure. Грозовая помеха в радио — шлюз накапливает 30 секунд пакетов, потом резко выдаёт. Голый in-memory pipe без блокировки на upstream получает OOM, когда downstream не справляется. `Pipe.channel` в miniFlink имеет явный capacity — producer блокируется когда capacity исчерпан. Сеть-шлюз дольше держит запрос, но сервис *не падает*.

Объединение потоков с выравниванием watermark’a. Газовый пайплайн потребляет три источника — raw RSSI, окна локации, газ. У каждого свой event-time и свои watermark’и. Если их просто “склеить” (`Stream.append`), один поток обгонит другие, и состояние обогатится событиями “из будущего”. `Mf_event.union` выравнивает watermark по минимуму из подключенных потоков — ровно та семантика которая нужна, и которую *очень неприятно* писать руками каждый раз.

Параллелизация задним числом. Самое заметное. Изначальный ex07 написан в однопоточковом виде. Добавление параллельной обработки — это одна обёртка `Parallel.run_parallel_simple` вокруг готового пайплайна, без изменения бизнес-логики; см. раздел 5, где показано как одна и та же `Pipelines.median_rssi` ускорила в 4.17 раз на 8 воркерах после такой обёртки. Голый код пришлось бы переписывать, разбираться с гонками, перепродумывать state-shared-vs-per-key. Здесь же исходник остался байт-в-байт тем же.

Тестируемость. Stream это просто `unit -> 'a Mf_event.t option`. Тест пайплайна — `Mf_event.of_list ~ts list` с явными временами, потом `Stream.to_list` результата, потом проверка инвариантов. Голый код, использующий системное время и сетевые сокет, потребовал бы моков или медленных интеграционных тестов. У нас наш сложный газовый retract-пайплайн покрыт 7-ю сценариями за <10ms прогона.

Это не «библиотечная польза вообще». Это *конкретные* места, где наивный аналог даёт либо потерю данных, либо нестабильный дашборд, либо невозможность тестов. Каждое исправлять руками дорого; пакет работающих абстракций в miniFlink обходится дешевле.

1.4 Что есть готового в miniFlink

Кратко, чтобы не повторяться в дальнейших разделах — ниже полный inventory того, что мы используем в ex07:

- `Pipe.event_time ~lateness` — вставка watermark’ов в поток по event-time;
- `Pipe.dedup` — дедупликация по rule-функции;
- `Pipe.window_agg_keyed` с `allowed_lateness` — скользящие окна с retract’ами при опоздавших пакетах;
- `Agg.median`, `Agg.group_by`, `Agg.top_k_by` — набор комбинируемых агрегатов;
- `Pipe.process_keyed` — per-key state с self-timers (для FSM состояния шахтёра);
- `Mf_event.union` — объединение потоков с выравниванием watermark’a по минимуму;

- `Pipe.materialize` — сворачивание retract-богатого потока в финальную таблицу записей (для sink-a).

Чего *нет* — это бизнес-логики и доменных типов. Их пишем мы; структура ниже как раз и есть рассказ о том, как мы их написали.

1.5 Что увидим дальше

Статья идёт от внешнего наружу. В разделе 2 разбираем устройство входов: почему два независимых потока вместо одного, какие в них аномалии. В разделе 3 — три пайплайна по очереди: FSM состояния, скользящие окна с медианой и интерполяцией позиции, retract-обогащение газовых алертов. Раздел 4 собирает наблюдения о retract-семантике как о сквозном механизме, проходящем через все три пайплайна. Раздел 5 приводит измерения производительности (на машине автора-эксперимента и на пользовательском i7-8700 с OCaml 5) и описывает направленные оптимизации, которые мы делали. Раздел 6 — что отложено, что не делалось намеренно, и при каких условиях стоит реализовать недостающее.

2 Входы

Этот раздел ещё не написан.

3 Три пайплайна

Этот раздел ещё не написан.

4 Retract как сквозной механизм

Этот раздел ещё не написан.

5 Производительность

Этот раздел ещё не написан.

6 Выводы

Этот раздел ещё не написан.